

PINESPHERE STAY

Offline-First Property Management System

USER & ROLE MANAGEMENT MODULE

Technical Blueprint, Database Design & Implementation Plan

Prepared from & aligned to the Pinesphere Stay Business Requirement Document (BRD)

Technology Stack: Flutter | FastAPI | PostgreSQL (Cloud) | SQLite / ObjectBox (Offline Local Store)

Table of Contents

1. Module Overview.....	4
1.1 Purpose & Scope.....	4
1.2 Objectives.....	4
1.3 Position Within the Overall System.....	4
2. User Hierarchy & Role Structure.....	6
2.1 Complete Hierarchy (as defined in the BRD).....	6
2.2 Role Catalogue.....	6
2.3 Role Responsibilities (per BRD).....	7
3. Roles & Permissions Matrix.....	9
3.1 Permission Catalogue.....	9
3.2 Recommended Role Matrix (module-mapped).....	9
4. Core Logics.....	11
4.1 Role-Based Access Control (RBAC) Enforcement Logic.....	11
4.2 Offline Authentication Logic.....	11
4.3 Single Primary Owner Enforcement Logic.....	11
4.4 Multi-Tenant User Scoping Logic.....	12
4.5 Custom Role Creation Logic (Scalability Hook).....	12
4.6 Session & Token Management Logic.....	12
4.7 Sync & Conflict Resolution Logic.....	12
4.8 Account Lockout & Security Logic.....	12
4.9 Staff Invitation & Onboarding Logic.....	13
4.10 Device-User Binding Logic.....	13
5. Workflows.....	14
5.1 Staff Creation Workflow.....	14
5.2 Staff Login Workflow (Online & Offline).....	14
5.3 Role Assignment / Change Workflow.....	14
5.4 Runtime Permission Check Workflow.....	15
5.5 Custom Role Creation Workflow.....	15
5.6 Staff Deactivation / Removal Workflow.....	15
5.7 Forgot PIN / Account Recovery Workflow.....	15
5.8 Sync Workflow for User & Role Data.....	16
5.9 Owner Transfer Workflow.....	16
6. Database Design.....	17
6.1 Entity-Relationship Diagram.....	17
6.2 Table Catalogue.....	17
6.3 Full Schema Definitions.....	18
6.4 Relationships Summary.....	22
6.5 Offline vs Cloud Strategy.....	22
6.6 Indexing & Performance.....	22
7. Dashboards - Count, Use Case & Fields.....	23

8. Implementation Plan..... 26

9. Scalability & Flexibility Considerations..... 28

10. Security Considerations (Module-Specific)..... 29

11. Summary..... 30

1. Module Overview

1.1 Purpose & Scope

The User & Role Management module is the security and access-control backbone of the Pinesphere Stay platform. It governs who can log in to the system, what each person is allowed to see and do, and how that access is created, changed, revoked, and audited — across the Super Admin Portal, the Property Owner App, and every staff login within a property. Because Pinesphere Stay is offline-first, this module must also work with zero internet connectivity: a receptionist must be able to log in, and the app must be able to enforce role permissions, even when the property's device has been offline for days.

This module does not duplicate business logic that belongs to other modules (Bookings, Payments, Housekeeping, etc.). Instead, it is the single source of truth for identity (who is this user) and authorization (what is this user allowed to do), which every other module queries before allowing an action.

1.2 Objectives

- Provide a secure, offline-capable login mechanism for Owners and Staff (PIN, biometric, password).
- Maintain a clear, hierarchical role structure exactly as defined in the BRD: Super Admin → Regional Manager / Sales / Support / Property → Property Owner → Manager / Reception / Accountant → Housekeeping / Maintenance / Restaurant → Guest.
- Enforce that every property has exactly one Primary Owner account, with all other logins created as Staff under that Owner.
- Support a fixed catalogue of system roles (Owner, Manager, Reception, Cashier, Housekeeping, Maintenance, Restaurant, Security, Auditor, Accountant) while remaining flexible enough to support custom, property-specific roles in future.
- Guarantee that role and permission data is fully available offline and synchronises safely (with conflict handling) whenever connectivity returns.
- Feed every user/role change into the platform-wide Audit Log module, since the BRD states nothing can be deleted from the audit log.
- Integrate cleanly with the Device Management module (device-user binding, remote logout/lock) and the Subscription module (plan limits on number of users/staff).

1.3 Position Within the Overall System

Per the BRD, Pinesphere is built as three applications sharing one multi-tenant FastAPI backend: the Super Admin Portal (Pinesphere operations), the Property Owner App (Flutter, offline-first), and the Guest Portal (WhatsApp / PWA, cloud-only). The User & Role Management module has a footprint in the first two:

- Super Admin Portal — manages Pinesphere-internal roles (Regional Manager, Sales Team, Support Team) and has global oversight of every property's users, plus the ability to remotely lock, log out, or revoke any user's access.
- Property Owner App — the Owner manages their own Staff (Manager, Reception, Housekeeping, Maintenance, Restaurant, Accountant, Cashier, Security, Auditor) entirely offline; changes sync to the cloud when internet is available.
- Guest Portal — Guests are not "users" of this module in the authentication sense (no password/PIN); they only receive secure, time-boxed WhatsApp links. Guest identity/consent is out of scope for this module and is handled by the Guest Portal / Booking modules.

Note: This document focuses exclusively on the User & Role Management module. Adjacent modules (Device Management, Subscription Management, Audit Logs, Property Onboarding) are referenced only where they intersect with users/roles (e.g. foreign keys), as per the current V1 module split you provided.

2. User Hierarchy & Role Structure

2.1 Complete Hierarchy (as defined in the BRD)

The BRD specifies one continuous chain of command from the Pinesphere company level down to the Guest. The diagram below is the authoritative structure this module implements.

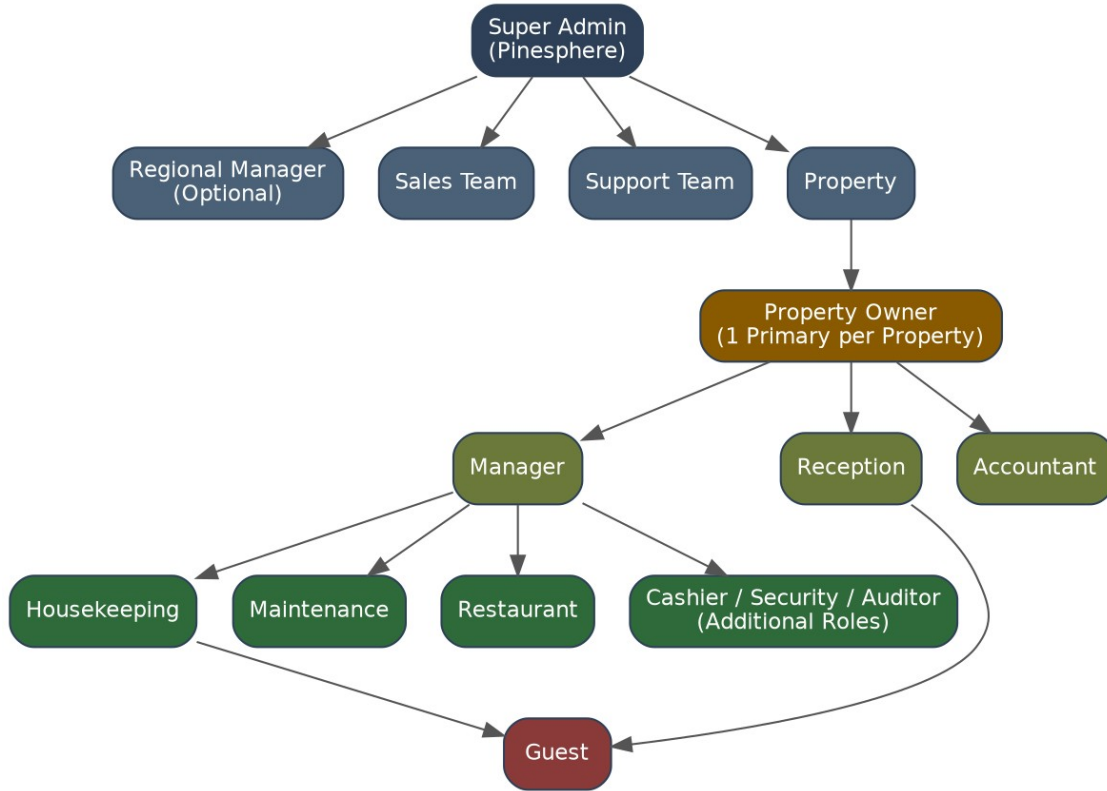


Figure 2.1 — Pinesphere Stay user hierarchy (Super Admin to Guest)

2.2 Role Catalogue

These are the fixed "system roles" shipped with V1. Every role below maps to exactly one row in the roles table (see Section 6) and is protected from deletion (`is_system_role = true`).

Tier	Role	Scope	Cardinality per Property
Pinesphere	Super Admin	Global — all properties	Fixed set, Pinesphere staff only
Pinesphere	Regional Manager (optional)	Group of properties	Optional, Pinesphere staff only
Pinesphere	Sales Team	Leads, demos, trial activation, conversion	Pinesphere staff only
Pinesphere	Support Team	Diagnostics, backups, password/license resets	Pinesphere staff only
Property	Owner (Primary)	Full control of one property	Exactly 1 per property
Property	Manager	Operational control under Owner	0..N
Property	Reception	Bookings, check-in/out, payments	0..N

Tier	Role	Scope	Cardinality per Property
Property	Accountant	Payments, expenses, GST, reports, invoices	0..N
Property	Cashier	Payment collection subset of Reception	0..N
Property	Housekeeping	Cleaning, laundry, room status, lost & found	0..N
Property	Maintenance	Complaint tracking, repair status	0..N
Property	Restaurant	Orders, kitchen status, billing, stock	0..N
Property	Security	Access/complaint monitoring (BRD role list)	0..N
Property	Auditor	Read-only oversight role (BRD role list)	0..N
Guest	Guest	No login credentials — WhatsApp / PWA link only	1 per booking

2.3 Role Responsibilities (per BRD)

Owner

- Create and manage staff
- Change room prices
- Manage the property's subscription
- Manage registered devices
- View all reports
- Approve refunds
- Manage settings

Reception

- Create bookings, check-in, check-out, collect payments, view rooms (allowed)
- Cannot: delete data, change subscription, manage users, export data

Housekeeping

- Cleaning, laundry, room status, lost and found only

Maintenance

- Broken furniture, electrical, plumbing, AC, TV, fan, complaint tracking

Restaurant

- Order taking, kitchen status, billing, stock (inventory deduction)

Accountant

- Payments, expenses, GST, reports, invoices

Sales Team (Pinesphere)

- Lead management, demo booking, quotation, trial activation, customer conversion, renewal reminders, commission, pipeline

Support Team (Pinesphere)

- View logs, view sync status, reset password/license, force sync, view errors, generate/restore backup, chat with customer, remote diagnostics

Design decision: "Cashier", "Security", and "Auditor" appear in the BRD's Staff Management role list but are not elaborated with dedicated feature sections. They are included as system roles with sensible default permission sets (Cashier ≈ payments-only subset of Reception; Security ≈ device/audit visibility; Auditor ≈ read-only across financial + operational modules) — kept intentionally narrow and adjustable through the Role & Permission Configuration Dashboard (Section 7).

3. Roles & Permissions Matrix

3.1 Permission Catalogue

The BRD lists these permission types under Staff Management, extended here with a few implied additions (marked *) so every module in your V1 list has a corresponding permission the Role & Permission engine can check.

Permission Code	Description	Access Levels Supported
BOOKINGS	Create / view / manage bookings	Full, Own, View, None
CHECKIN_CHECKOUT	Perform check-in and check-out	Full, View, None
PAYMENTS	Collect and view payments	Full, Limited, View, None
EXPENSES	Record and view expenses	Full, View, None
REPORTS	View operational/financial reports	Full (Global), Operational, Limited (Financial), None
DELETE	Delete records across modules	Allowed, Denied
REFUND	Approve refunds	Allowed, Denied
ROOM_PRICING	Change room / seasonal pricing	Allowed, Denied
DISCOUNT	Apply discounts / coupons	Allowed, Denied
USERS	Create/edit/deactivate staff & assign roles	Full, Limited, View, None
INVENTORY	Manage stock, vendors, purchase entries	Full, View, None
RESTAURANT	Orders, kitchen, billing, stock	Full, View, None
SETTINGS	Property-level configuration	Full, None
DEVICE_MANAGEMENT*	Approve / lock / revoke devices	Full, View, None
AUDIT_LOGS	View the audit trail	Full (Global), Property Only, Limited (Own actions), None
SUBSCRIPTION*	View/renew subscription	Full, View, None
HOUSEKEEPING_MAINTENANCE*	Cleaning/maintenance task status	Full, Update Status, None

3.2 Recommended Role Matrix (as specified in the BRD, module-mapped)

This is the exact matrix from the BRD's "Recommended Role Matrix", restated per module so it can be seeded directly into the role_permissions table (Section 6) as default data at deployment time.

Module	Super Admin	Owner	Manager	Reception	Housekeeping	Accountant	
Property Onboarding	Full	—	—	—	—	—	—
Subscription Management	Full	View / Renew	—	—	—	—	—
User & Role Management	Full	Full	Limited	—	—	—	—
Device Management	Full	View	—	—	—	—	—
Room Management	View	Full	Full	View	Limited	—	View

Module	Super Admin	Owner	Manager	Reception	Housekeeping	Accountant	
					(status)		
Booking Management	View	Full	Full	Create/ Manage	—	—	Own
Check-in / Check-out	View	Full	Full	Full	—	—	Digi in
Payments	View	Full	Full	Limited	—	Collect	Pay
Reports	Global	Full	Operational	Limited	—	Financial	Own
Audit Logs	Full	Property Only	Limited	Own Actions	Own Actions	Financial	—

Reading the matrix for User & Role Management

- Super Admin: full CRUD on users/roles across every property, plus remote lock/deactivate.
- Owner: full CRUD on staff and roles within their own property only (property-scoped).
- Manager: limited — can view staff, raise requests (e.g. leave, task assignment) but cannot create/delete users or change roles.
- Reception / Housekeeping / Accountant / Guest: no access to this module at all (they only see their own profile screen, covered as a dashboard in Section 7).

Access levels are stored as an enum on `role_permissions.access_level` (FULL, LIMITED, VIEW, OWN, NONE) rather than a boolean, so the same permissions table can express the graded access the BRD describes (e.g. "Limited", "Own Actions", "Financial") without needing a separate table per access tier.

4. Core Logics

This section defines the exact business logic the Flutter app and FastAPI backend must implement for this module. Each logic block is written so it can be translated almost directly into function/service names during development.

4.1 Role-Based Access Control (RBAC) Enforcement Logic

Every screen, button, and API endpoint is guarded by a single reusable check rather than scattered if/else conditions:

- `hasPermission(userId, permissionCode, requiredLevel)` is the single choke-point function, implemented identically on the Flutter client (for UI hide/show + offline enforcement) and the FastAPI backend (for hard server-side enforcement).
- On login (online or offline), the full `role_permissions` snapshot for the user's role is loaded once into an in-memory `PermissionSet` and cached locally — this avoids a DB/query round trip on every tap.
- The client uses the cached `PermissionSet` to enable/disable UI (e.g. grey out "Delete Booking" for Reception); the server is the final authority and re-checks on every API call — the client-side check is a UX convenience, never the security boundary.
- Access levels are ordered (`NONE < VIEW < OWN < LIMITED < FULL`) so a single comparison (`userLevel >= requiredLevel`) covers most checks; special cases (e.g. "OWN" meaning "only records created by me") add a row-level filter (`created_by = current_user_id`) on top of the level check.

4.2 Offline Authentication Logic

Because the app must run with zero connectivity, authentication cannot depend on a live call to the server at every login.

- First login for any device must happen online (or via a Support-generated activation code) — this is when the credential hash, role, and permission snapshot are downloaded and cached locally.
- Subsequent logins check credentials against the locally stored, encrypted hash: PIN (4–6 digit) hashed with a salted algorithm (e.g. Argon2/bcrypt) or biometric assertion validated via the OS keystore/secure enclave — never plain-text PIN storage.
- A password/PIN is never transmitted or re-validated against the server on every login; the server is only re-contacted opportunistically (background sync) to detect remote revocations.
- If the cached credential set is missing or corrupted (e.g. app reinstalled with no local backup) and there is no internet, login is denied with a clear "connect to the internet once to restore access" message rather than a silent failure.

4.3 Single Primary Owner Enforcement Logic

- Each property may have exactly one user row with `is_primary_owner = true`. This is enforced at the database layer with a partial unique index (unique on `property_id` where `is_primary_owner = true`), not only in application code.
- Ownership can move (BRD: "Transfer Ownership", performed by Super Admin) via a dedicated `transfer_ownership(propertyId, fromUserId, toUserId)` operation that, in a single transaction, flips `is_primary_owner` off the old owner and on for the new one, and writes two `audit_logs` entries.
- The primary Owner account can never be deleted directly — only deactivated as part of a Transfer Ownership or Property Offboarding operation initiated by Super Admin.

4.4 Multi-Tenant User Scoping Logic

- Every query for users, roles (custom ones), and sessions is automatically scoped by `property_id` at the repository layer (both in the FastAPI service layer and the local SQLite/ObjectBox repository) — a developer should not be able to accidentally query across properties.
- System roles (Owner, Manager, Reception, etc.) are global rows with `property_id` = NULL and are shared/read-only reference data; only custom, property-specific roles carry a `property_id`.
- Pinesphere Super Admin, Regional Manager, Sales and Support roles are the only ones allowed a NULL `property_id` on the users table itself (i.e., a login not tied to any single property).

4.5 Custom Role Creation Logic (Scalability Hook)

- V1 ships only the fixed system roles from Section 2, but the roles/role_permissions schema already supports property-scoped custom roles so this can be turned on later without a schema migration.
- Logic: an Owner (with `USERS` = FULL) may clone an existing system role into a new custom role row (`is_system_role` = false, `property_id` = <own property>), then adjust individual permission rows in `role_permissions`.
- Custom roles are always evaluated with the same `hasPermission()` function as system roles — no separate code path — which keeps the feature additive rather than a fork.

4.6 Session & Token Management Logic

- Online session: FastAPI issues a short-lived JWT access token + longer-lived refresh token on successful `/auth/login`; refresh happens silently in the background when the app is online.
- Offline session: a locally-signed offline session token is created after successful local PIN/biometric verification, valid until the earlier of (a) a configurable max-offline-duration (e.g. 30 days, configurable per Subscription plan) or (b) the next successful online sync.
- Every session is recorded in `user_sessions` with the originating `device_id`, so Device Management and Super Admin can see exactly which device is logged in as which user.
- Logout (manual or remote, from Super Admin's "Remote Device Control") immediately invalidates the session row; on the next sync/heartbeat the device honours the remote logout even if it is mid-offline-session.

4.7 Sync & Conflict Resolution Logic for User/Role Data

- All local writes to users, roles, `role_permissions` go through a generic outbox pattern: the write is applied locally immediately, and a corresponding row is appended to `user_sync_log` with `status` = PENDING.
- When connectivity returns, the Synchronization Engine (shared platform service, per BRD) pushes pending rows, then pulls server-side changes (new roles, permission changes made from Super Admin, remote deactivations).
- Conflict rule for this module specifically: Owner/Super Admin edits always win over Staff-originated edits (Staff cannot edit their own role/permissions in the first place, so true conflicts are rare); for concurrent Owner edits from two devices, Last-Write-Wins by `updated_at` timestamp, with the losing change preserved in `audit_logs` for traceability rather than silently discarded.
- A locally created staff account that has not yet synced is flagged `is_pending_sync` = true in the UI so the Owner knows that account will not yet appear on other devices or in Super Admin.

4.8 Account Lockout & Security Logic

- Configurable max failed PIN/biometric attempts (default 5) locks the account locally (`status` = LOCKED) and logs a `SECURITY_ALERT` entry in `audit_logs`.

- A locked account can be unlocked by: (a) the Owner from the Staff Dashboard, (b) Support/Super Admin remotely, or (c) automatically after a cool-down window, per configuration.
- Repeated lockouts across multiple staff on the same device within a short window raise a fraud/unusual-activity flag surfaced on the Security & Audit Dashboard (Section 7.6) and to the AI Features' "Fraud and unusual activity alerts".

4.9 Staff Invitation & Onboarding Logic

- Owner creates a staff_invitations row (mobile number + role) rather than a live user directly; an activation code / WhatsApp link (reusing the platform's WhatsApp Integration) is generated for the staff member to set their own PIN on first login.
- If created while offline, the invitation is queued exactly like any other write (Section 4.7) and the WhatsApp message is actually sent only once the device/server syncs and has connectivity.
- Invitation expires after a configurable window; expired, unused invitations do not create a user row and are periodically cleaned up.

4.10 Device–User Binding Logic (integration with Device Management)

- A user_devices row links a user to one or more registered devices; the BRD's Device Management screen ("Current User", "Last Login") reads directly from this table rather than duplicating device/user linkage elsewhere.
- When Super Admin performs "Revoke License → Logout → Erase Local Database → Disable Login" for a resigned staff member's device, this module's response is: invalidate all user_sessions rows for that device, set the user's status to INACTIVE if it was their only device, and write the mandatory audit_logs entry.

5. Workflows

Step-by-step flows for every user/role scenario, written the same linear "A → B → C" style used throughout the BRD.

5.1 Staff Creation Workflow

1. Owner opens Staff Dashboard → taps "Add Staff"
2. Owner enters: Name, Mobile Number, Role, (optional) Email, Photo
3. System checks Subscription plan's Max Staff / Max Users limit
4. If within limit → create staff_invitations row (or user row directly if offline-first-account creation is enabled)
5. WhatsApp invitation link sent to staff mobile (queued if offline)
6. Staff opens link → sets PIN / enables biometric → account activated
7. Event written to audit_logs ("Owner created Staff: <name>, Role: <role>")

5.2 Staff Login Workflow (Online & Offline)

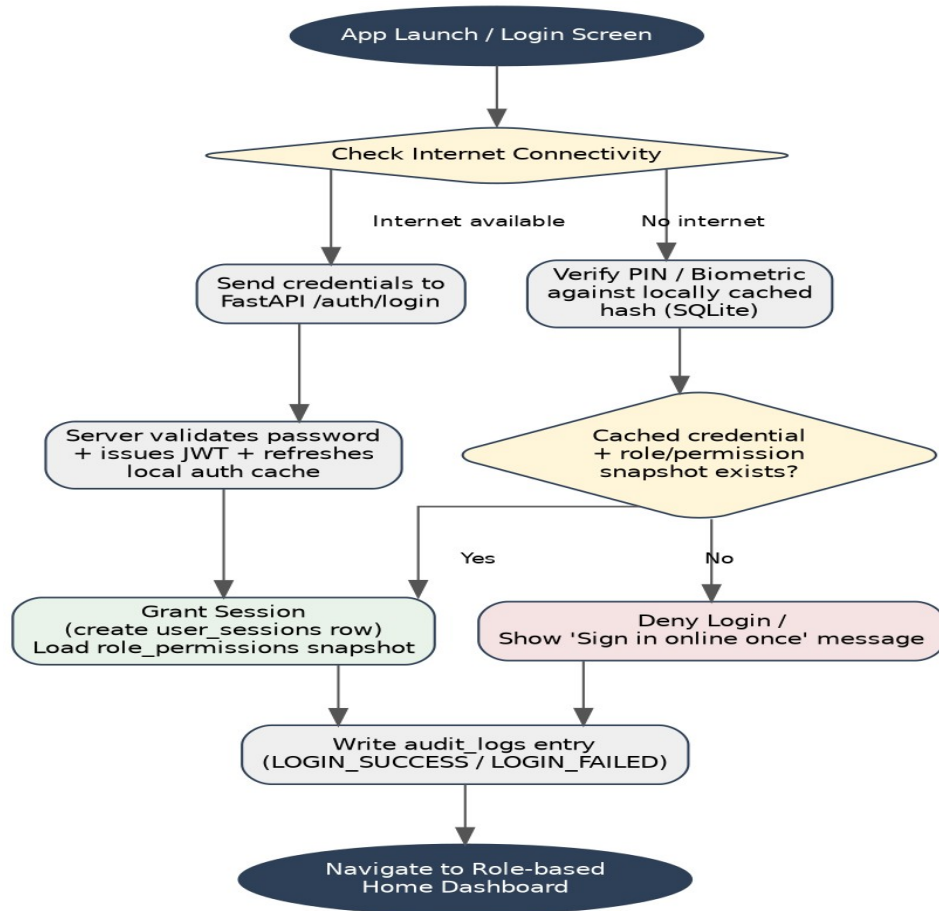


Figure 5.2 — Login decision flow covering both online and fully offline paths

5.3 Role Assignment / Change Workflow

8. Owner opens a staff member's profile from the Staff Dashboard
9. Owner selects a new Role from the system role list (or a custom role, if enabled)
10. System re-evaluates the user's cached PermissionSet immediately on the same device

11. Change queued for sync (user_sync_log) if offline; pushed immediately if online
12. Audit log entry written: "Role changed: <old role> → <new role>"
13. On the staff member's own device, the next successful sync (or next login) refreshes their local PermissionSet — UI updates automatically

5.4 Runtime Permission Check Workflow

14. User attempts an action (e.g. tap "Delete Booking")
15. Client calls hasPermission(userId, 'DELETE', requiredLevel) against the cached PermissionSet
16. If denied locally → action blocked immediately, no network call made
17. If allowed locally → request proceeds to FastAPI, which independently re-validates the same permission server-side before committing the change
18. Any server-side denial (e.g. stale/cached permission on the device) returns a 403 and forces an immediate permission-set refresh on the client

5.5 Custom Role Creation Workflow (future-ready, schema supports it in V1)

19. Owner opens Role & Permission Configuration Dashboard
20. Owner selects "Clone Role" on an existing system role (e.g. Reception)
21. System creates a new roles row scoped to that property (is_system_role = false)
22. Owner edits individual permission rows via the Permission Matrix editor
23. New role becomes selectable when creating/editing staff for that property only

5.6 Staff Deactivation / Removal Workflow

24. Owner (or Super Admin) selects a staff member → "Deactivate"
25. System sets users.status = INACTIVE and invalidates all active user_sessions for that user
26. Any devices bound to the user (user_devices) are flagged for logout on next connectivity
27. Historical records created by that staff member (bookings, payments, etc.) are preserved and remain attributed to them — only login access is removed
28. Audit log entry written; if the user was the sole account on a device, the device shows a "no active user" state until reassigned

5.7 Forgot PIN / Account Recovery Workflow

29. Staff member selects "Forgot PIN" on login screen
30. If online: OTP sent via SMS/WhatsApp to the registered mobile → staff sets a new PIN → credential_reset_requests row marked APPROVED
31. If offline: staff must ask the Owner (who is authorised) to reset the PIN locally from the Staff Dashboard, which creates a local reset entry that syncs later
32. If the Owner themselves is locked out: only Support/Super Admin (from the Support Portal) can perform the reset, per the BRD's Support Portal capability "Reset Password"
33. Every reset, however initiated, is written to audit_logs with who approved it

5.8 Sync Workflow for User & Role Data

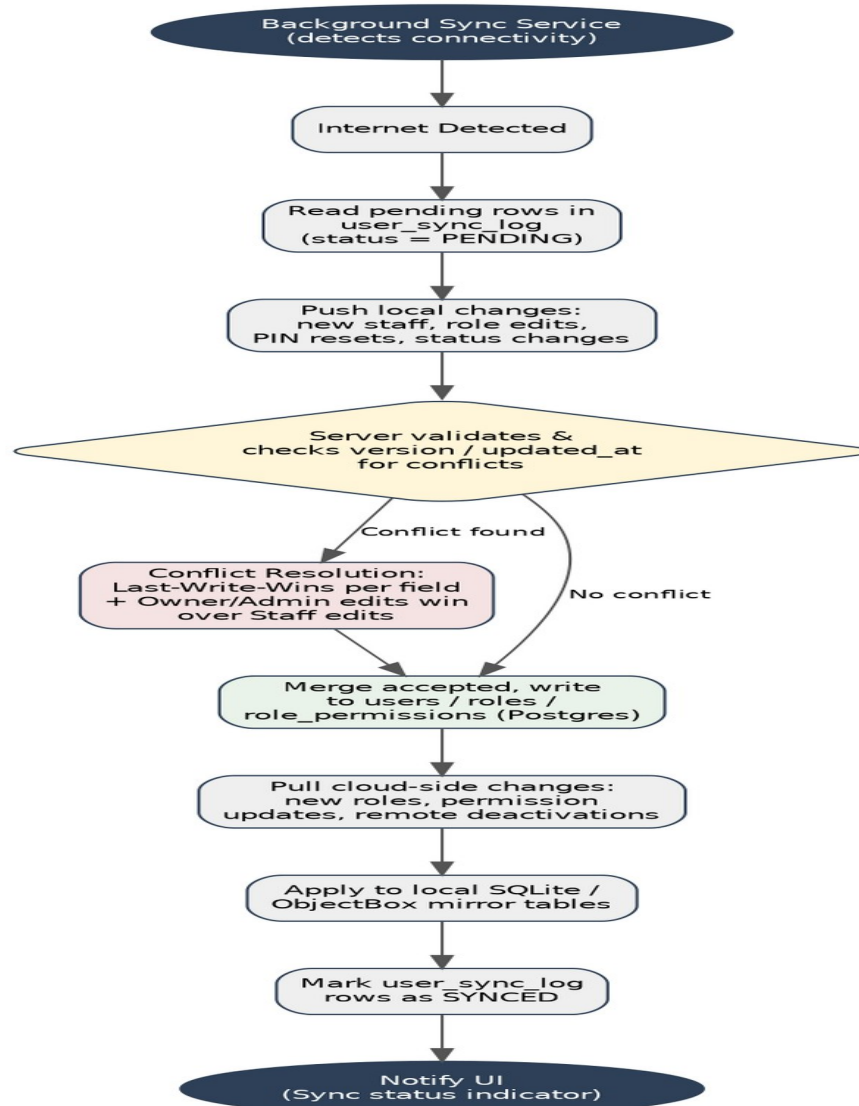


Figure 5.8 — Background sync and conflict-resolution flow for user/role changes

5.9 Owner Transfer Workflow

34. Super Admin opens the property record → "Transfer Ownership"
35. Super Admin selects the existing Owner and the new Owner (must already exist as a Staff account, or is created fresh)
36. System runs `transfer_ownership()` in a single transaction: unsets `is_primary_owner` on the old user, sets it on the new user
37. Old owner is automatically demoted to a Manager-equivalent role (configurable) or deactivated, per the Super Admin's choice
38. Two `audit_logs` entries are written (one per user affected) and the change is pushed to the property's devices on next sync

Table	Purpose	Owned By This Module?
user_devices	Links a user to registered device(s)	Yes (links to Device Management's devices table)
user_sessions	Active/expired login sessions, online & offline	Yes
staff_invitations	Pending staff onboarding invites	Yes
credential_reset_requests	PIN/password reset requests & approvals	Yes
user_sync_log	Outbox of pending offline→cloud changes for this module	Yes
audit_logs	Immutable action trail	No — owned by Audit Logs module; written to by this module

6.3 Full Schema Definitions

6.3.1 roles

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
property_id	BIGINT / UUID, NULL	FK → properties.id; NULL for global system roles
role_code	VARCHAR(40)	NOT NULL — e.g. OWNER, MANAGER, RECEPTION
role_name	VARCHAR(80)	NOT NULL
is_system_role	BOOLEAN	NOT NULL, DEFAULT true
description	TEXT	NULL
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

- Constraint: UNIQUE(property_id, role_code) — same role code cannot repeat within one property's custom roles.
- Constraint: CHECK (is_system_role = true AND property_id IS NULL) OR (is_system_role = false AND property_id IS NOT NULL) — system roles are always global; custom roles are always property-scoped.

6.3.2 permissions

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
permission_code	VARCHAR(60)	NOT NULL, UNIQUE — e.g. BOOKINGS, PAYMENTS, USERS
module_name	VARCHAR(60)	NOT NULL — which V1 module this permission gates
description	TEXT	NULL

6.3.3 role_permissions

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
role_id	BIGINT / UUID	FK → roles.id, NOT NULL, ON DELETE CASCADE
permission_id	BIGINT / UUID	FK → permissions.id, NOT NULL, ON DELETE CASCADE
access_level	ENUM	NOT NULL — FULL, LIMITED, VIEW, OWN, NONE
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

- Constraint: UNIQUE(role_id, permission_id) — one grant row per role/permission pair.

6.3.4 users

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
property_id	BIGINT / UUID, NULL	FK → properties.id; NULL for Pinesphere-internal users
role_id	BIGINT / UUID	FK → roles.id, NOT NULL
name	VARCHAR(120)	NOT NULL
mobile_number	VARCHAR(15)	NOT NULL, UNIQUE per property_id
email	VARCHAR(120)	NULL, UNIQUE where NOT NULL
username	VARCHAR(60)	NULL, UNIQUE per property_id
password_hash	VARCHAR(255)	NULL — used for Super Admin / web logins
pin_hash	VARCHAR(255)	NULL — used for mobile offline login
biometric_enabled	BOOLEAN	NOT NULL, DEFAULT false
is_primary_owner	BOOLEAN	NOT NULL, DEFAULT false
status	ENUM	NOT NULL, DEFAULT ACTIVE — ACTIVE, INACTIVE, LOCKED, SUSPENDED
failed_login_attempts	SMALLINT	NOT NULL, DEFAULT 0
profile_photo_url	TEXT	NULL
created_by	BIGINT / UUID	FK → users.id, NULL (self-referencing)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
is_pending_sync	BOOLEAN	NOT NULL, DEFAULT false — local-store only flag

- Constraint: UNIQUE partial index on (property_id) WHERE is_primary_owner = true AND status <> 'INACTIVE' — enforces exactly one active primary Owner per property (Section 4.3).
- Constraint: CHECK (password_hash IS NOT NULL OR pin_hash IS NOT NULL) — a user must have at least one credential set.
- Constraint: UNIQUE(property_id, mobile_number) and UNIQUE(property_id, username) — uniqueness is scoped per property, so the same phone number can (rarely) be staff at two different properties without conflict.

6.3.5 user_devices

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
user_id	BIGINT / UUID	FK → users.id, NOT NULL, ON DELETE CASCADE
device_id	BIGINT / UUID	FK → devices.id (Device Management module), NOT NULL
is_primary_device	BOOLEAN	NOT NULL, DEFAULT true
linked_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
status	ENUM	NOT NULL — ACTIVE, REVOKED

- Constraint: UNIQUE(user_id, device_id) — no duplicate linkage rows.

6.3.6 user_sessions

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
user_id	BIGINT / UUID	FK → users.id, NOT NULL, ON DELETE CASCADE
device_id	BIGINT / UUID	FK → devices.id, NOT NULL
session_token	VARCHAR(255)	NOT NULL, UNIQUE
is_offline_session	BOOLEAN	NOT NULL, DEFAULT false
issued_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
expires_at	TIMESTAMPTZ	NOT NULL
revoked_at	TIMESTAMPTZ	NULL
revoked_reason	VARCHAR(120)	NULL — e.g. REMOTE_LOGOUT, LOCKOUT, MANUAL

6.3.7 staff_invitations

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
property_id	BIGINT / UUID	FK → properties.id, NOT NULL
role_id	BIGINT / UUID	FK → roles.id, NOT NULL
invited_by	BIGINT / UUID	FK → users.id, NOT NULL
mobile_number	VARCHAR(15)	NOT NULL
invitation_token	VARCHAR(255)	NOT NULL, UNIQUE
status	ENUM	NOT NULL, DEFAULT PENDING — PENDING, ACCEPTED, EXPIRED, CANCELLED
expires_at	TIMESTAMPTZ	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

6.3.8 credential_reset_requests

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
user_id	BIGINT / UUID	FK → users.id, NOT NULL, ON DELETE CASCADE
reset_type	ENUM	NOT NULL — PIN, PASSWORD, BIOMETRIC_RESET
requested_via	ENUM	NOT NULL — OTP, OWNER_APPROVAL, SUPPORT_PORTAL
approved_by	BIGINT / UUID	FK → users.id, NULL
status	ENUM	NOT NULL, DEFAULT PENDING — PENDING, APPROVED, REJECTED, EXPIRED
requested_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
resolved_at	TIMESTAMPTZ	NULL

6.3.9 user_sync_log (local-store outbox; mirrored to cloud for traceability)

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
entity_type	VARCHAR(40)	NOT NULL — USER, ROLE, ROLE_PERMISSION
entity_id	BIGINT / UUID	NOT NULL
operation	ENUM	NOT NULL — CREATE, UPDATE, DELETE
payload	JSONB	NOT NULL — snapshot of the change
sync_status	ENUM	NOT NULL, DEFAULT PENDING — PENDING, SYNCED, CONFLICT, FAILED
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
synced_at	TIMESTAMPTZ	NULL

6.3.10 audit_logs (reference — owned by the Audit Logs module)

This module writes to, but does not own, the audit_logs table. Minimum columns this module depends on:

Column	Type	Constraints
id	BIGINT / UUID	PRIMARY KEY
user_id	BIGINT / UUID	FK → users.id — actor who performed the action
property_id	BIGINT / UUID, NULL	FK → properties.id
module_name	VARCHAR(60)	NOT NULL — "USER_ROLE_MANAGEMENT"
action	VARCHAR(120)	NOT NULL — e.g. STAFF_CREATED, ROLE_CHANGED, LOGIN_FAILED
old_value	JSONB	NULL
new_value	JSONB	NULL

Column	Type	Constraints
created_at	TIMESTAMPZ	NOT NULL, DEFAULT now()

Per the BRD: "Nothing can be deleted from the audit log." audit_logs therefore has no DELETE permission granted to any role, and no UPDATE either — it is append-only at the database-permission level, not just the application level.

6.4 Relationships Summary

- properties (1) — (N) roles [custom roles only]
- properties (1) — (N) users
- roles (1) — (N) role_permissions — (N) — (1) permissions
- roles (1) — (N) users
- users (1) — (N) users [self-referencing via created_by]
- users (1) — (N) user_devices — (N) — (1) devices [Device Management]
- users (1) — (N) user_sessions
- properties (1) — (N) staff_invitations — (N) — (1) roles
- users (1) — (N) credential_reset_requests
- users (1) — (N) audit_logs
- users (1) — (N) user_sync_log

6.5 Offline (SQLite/ObjectBox) vs Cloud (PostgreSQL) Strategy

- All tables in Section 6.3 exist in both stores except user_sync_log's cloud mirror (used for audit/traceability only) and any Pinesphere-internal-only rows (Super Admin/Sales/Support users are not replicated to property devices at all).
- Local IDs are generated client-side as UUIDs (not auto-increment integers) so records created offline never collide with cloud-generated IDs from another device.
- Sensitive columns (pin_hash, password_hash) are stored encrypted at rest locally using the device's secure keystore-backed encryption key, per the BRD's "Local database encryption" security requirement.
- ObjectBox is used where relational joins are simple and read-speed for permission checks matters most (users, roles, role_permissions, permissions); SQLite is acceptable for lower-frequency tables (staff_invitations, credential_reset_requests, user_sync_log) — final choice is an implementation detail behind a repository interface, so it can change without affecting the rest of the app.

6.6 Indexing & Performance

- INDEX on users(property_id, status) — used by every Staff Dashboard listing query.
- INDEX on users(mobile_number) — used for login lookup and duplicate-guest/staff detection.
- INDEX on role_permissions(role_id) — used on every permission-set load at login.
- INDEX on user_sessions(user_id, expires_at) — used to quickly find/expire active sessions.
- INDEX on user_sync_log(sync_status, created_at) — used by the background Sync Engine to fetch the pending outbox in order.
- INDEX on audit_logs(property_id, created_at DESC) — used by the Audit Logs module dashboards; kept here since this module is a heavy writer to that table.

7. Dashboards — Count, Use Case & Fields

The User & Role Management module surfaces through six distinct dashboards, one for each level of the hierarchy that has a genuine need to view or manage user/role data. Each is a separate screen in Flutter (Owner App) or the Super Admin Portal, backed by its own FastAPI read endpoint.

7.1 Dashboard Inventory

#	Dashboard	Audience	Primary Use Case
1	Global User & Access Analytics Dashboard	Super Admin	Cross-property oversight of users, roles, security events
2	Staff & Role Management Dashboard	Property Owner	Day-to-day staff CRUD, the module's main screen
3	Role & Permission Configuration Dashboard	Property Owner	Configure/clone roles and edit the permission matrix
4	Team Overview Dashboard	Manager	Limited, read-mostly view of their own team
5	My Profile & Access Dashboard	Any Staff (self-service)	View own role/permissions, manage own credentials
6	User Security & Audit Dashboard	Super Admin / Support / Auditor	Failed logins, lockouts, device anomalies, permission-change trail

7.2 Dashboard 1 — Global User & Access Analytics Dashboard (Super Admin)

Use case: gives Pinesphere operations a single screen to understand how many users exist across the platform, how healthy logins/devices are, and where to intervene.

Fields

- Total Users (platform-wide)
- Users by Role (breakdown chart: Owner / Manager / Reception / etc.)
- Active vs Inactive vs Locked Users
- New Staff Created (Today / This Week / This Month)
- Properties with No Active Owner (data-integrity alert)
- Failed Login Attempts (Today)
- Locked Accounts (current count, drill-down list)
- Pending Staff Invitations (not yet accepted)
- Devices per User (avg.) / Users with 0 Registered Devices
- Recent Role Changes (last N, with who/when)
- Pending Sync Count (users/roles not yet synced from properties)
- Search / Filter by Property, Role, Status

7.3 Dashboard 2 — Staff & Role Management Dashboard (Owner)

Use case: the Owner's home screen for this module — list, add, edit, deactivate staff, and see who's currently active. Matches the BRD's Staff Management feature list directly.

Fields

- Staff List (Name, Photo, Role, Status, Last Login, Assigned Device)
- "Add Staff" action button (opens Staff Creation Workflow, Section 5.1)
- Quick Filters: All / Active / Inactive / Locked / Pending Invitation
- Search by Name / Mobile Number
- Per-row Quick Actions: Edit, Change Role, Reset PIN, Deactivate, View Activity
- Staff Count vs Subscription Plan Limit (e.g. "6 / 10 staff used")
- Attendance Snapshot (link-through to Staff Management module's Attendance feature)
- Pending Invitations widget (mobile number, role, expiry, resend action)

7.4 Dashboard 3 — Role & Permission Configuration Dashboard (Owner)

Use case: lets the Owner see and, where allowed, adjust what each role can do — the visual equivalent of the Role Matrix in Section 3.2, scoped to their own property.

Fields

- Role List (system roles + any custom roles for this property)
- Permission Matrix Grid (Roles as rows, Permissions as columns, access level as cell value)
- "Clone Role" action (creates a custom role, Section 5.5)
- Per-permission Access Level selector (Full / Limited / View / Own / None) — disabled for protected system-role/permission combinations (e.g. Owner's USERS permission cannot be lowered below Full)
- Number of Staff Currently on Each Role (so the Owner sees the blast radius before changing a role)
- Change History for this role (last N changes, who made them)

7.5 Dashboard 4 — Team Overview Dashboard (Manager)

Use case: gives a Manager operational visibility into their team without exposing account-management actions reserved for the Owner (per the BRD matrix: Manager = "Limited" on User & Role Management).

Fields

- My Team List (Name, Role, Status, Currently Clocked-in/Attendance state)
- Task Assignment Snapshot (links to Staff Management's Task Assignment)
- Shift / Attendance Summary for the team
- Read-only Role labels per team member (no edit control)
- Raise Request button (e.g. request Owner to add/deactivate a staff member — creates an approval item for the Owner, does not directly change data)

7.6 Dashboard 5 — My Profile & Access Dashboard (Self-Service, all Staff)

Use case: every logged-in user (Owner or Staff) can see their own identity, role, and permission set, and manage their own login credentials — without needing Owner intervention for routine tasks.

Fields

- My Details (Name, Mobile, Email, Photo, Role)
- My Permissions (read-only list of what I can/can't do, in plain language)

- My Registered Device(s) (device name, last login)
- Change PIN / Enable-Disable Biometric Login
- My Recent Activity (own actions only, from audit_logs filtered to Own Actions per the Role Matrix)
- Sync Status Indicator (Last Synced At / Pending Changes Count)

7.7 Dashboard 6 — User Security & Audit Dashboard (Super Admin / Support / Auditor)

Use case: a focused security lens over the same data — separate from Dashboard 1's broad analytics — so Support engineers and Auditors can investigate incidents quickly, matching the BRD's Support Portal ("View Logs", "Reset License", "Remote Diagnostics") and Security section ("Fraud and unusual activity alerts").

Fields

- Failed Login Attempts (by user, by device, with timestamps)
- Currently Locked Accounts (with unlock action for Support)
- Recent Permission / Role Changes (full audit trail, filterable by property/user/date)
- Device Anomalies (e.g. same user logged in from 2 devices simultaneously, unusual location)
- Remote Session Control (force logout / revoke session for any user-device pair)
- Credential Reset Requests Queue (pending, approved, rejected)
- Suspicious Activity Flags (auto-generated, e.g. repeated lockouts in a short window)

Guests are intentionally excluded from this list: per the BRD, Guests never hold a users row with credentials — they interact only through time-boxed WhatsApp/PWA links owned by the Guest Portal and Booking modules, so no guest-facing dashboard belongs to User & Role Management.

8. Implementation Plan

Organised by phase rather than by date, so your team can sequence work at its own pace. Each phase is independently shippable and builds on the previous one.

8.1 Phase 1 — Core Identity & Offline Login

- DB: create properties (stub/reference), roles, permissions, role_permissions, users tables; seed system roles + permission catalogue + default Role Matrix (Section 3.2).
- FastAPI: /auth/login, /auth/refresh, /users (CRUD scoped to property), basic RBAC middleware (hasPermission dependency).
- Flutter: Login screen (password path for first-time online setup), local encrypted storage for PIN/biometric, PermissionSet in-memory cache, Owner's Staff Dashboard (list + create + edit, online only for this phase).

8.2 Phase 2 — True Offline-First & Multi-Tenant Hardening

- DB: add user_sessions, user_sync_log, staff_invitations, credential_reset_requests.
- FastAPI: sync endpoints (push/pull), invitation issuing (+ WhatsApp integration hook), conflict-resolution logic (Section 4.7).
- Flutter: offline PIN/biometric login path, local outbox writer, background Sync Engine integration, Staff Invitation flow end-to-end, PIN reset flows (online OTP + offline Owner-approval).

8.3 Phase 3 — Roles, Permissions UI & Dashboards

- DB: add support for custom roles (is_system_role = false path), indexes from Section 6.6.
- FastAPI: role-cloning endpoint, permission-matrix read/update endpoints, dashboard aggregation endpoints (Dashboards 1–6 in Section 7).
- Flutter: Role & Permission Configuration Dashboard, Team Overview Dashboard (Manager), My Profile & Access Dashboard (self-service), device-linked info surfaced from Device Management.

8.4 Phase 4 — Super Admin, Security & Analytics

- FastAPI/Portal: Global User & Access Analytics Dashboard, User Security & Audit Dashboard, remote lock/logout/revoke endpoints, Owner Transfer workflow.
- Security hardening: account lockout logic, fraud/unusual-activity flags, audit_logs write-only DB permissions, encryption-at-rest verification for local stores.
- Load/scale testing of the sync engine under many concurrent properties; index tuning per Section 6.6.

8.5 FastAPI Endpoint Reference

Method & Path	Purpose	Typical Caller
POST /auth/login	Online credential login, issues JWT	All roles
POST /auth/offline-bootstrap	Downloads credential hash + permission snapshot for offline use	Owner / Staff (first login)
POST /auth/refresh	Refreshes access token	All roles
POST /auth/logout	Invalidates a session	All roles
GET /users	List users (property-scoped, or global for Super	Owner, Manager

Method & Path	Purpose	Typical Caller
	Admin)	(limited), Super Admin
POST /users	Create a staff user / invitation	Owner, Super Admin
PATCH /users/{id}	Edit staff details / role / status	Owner, Super Admin
POST /users/{id}/deactivate	Deactivate a user	Owner, Super Admin
POST /users/{id}/reset-credential	Trigger PIN/password reset	Owner, Support, Super Admin
POST /users/transfer-ownership	Transfer primary Owner	Super Admin only
GET /roles	List system + custom roles for a property	Owner, Manager (view), Super Admin
POST /roles/clone	Create a custom role from a system role	Owner
GET /roles/{id}/permissions	Fetch permission matrix for a role	Owner, Super Admin
PATCH /roles/{id}/permissions	Update access levels for a role	Owner (own property), Super Admin
POST /devices/{id}/revoke	Remote revoke a device's session(s)	Super Admin, Support
GET /dashboards/user-analytics	Data for Dashboard 1	Super Admin
GET /dashboards/security-audit	Data for Dashboard 6	Super Admin, Support, Auditor
POST /sync/push	Push local outbox (users/roles/role_permissions changes)	Owner App (background)
GET /sync/pull	Pull cloud-side changes since last sync	Owner App (background)

8.6 Flutter App Structure (Offline-First, Repository Pattern)

Recommended module layout inside the Flutter app so this module stays cleanly separated and testable:

- lib/features/user_role_management/data/local/ — SQLite/ObjectBox DAOs for users, roles, permissions, sessions
- lib/features/user_role_management/data/remote/ — FastAPI client (Dio/Retrofit) for auth, users, roles endpoints
- lib/features/user_role_management/data/repository/ — UserRepository, RoleRepository — single interface the rest of the app calls; internally decides local vs remote and writes to the sync outbox
- lib/features/user_role_management/domain/ — PermissionSet, hasPermission(), entities (User, Role, Permission)
- lib/features/user_role_management/presentation/ — the six dashboards from Section 7 as separate screens/widgets, plus Login screen
- lib/core/sync/ — shared Synchronization Engine used by this and every other offline-first module (Bookings, Payments, etc.)
- lib/core/security/ — secure storage wrapper, biometric wrapper, encryption helpers shared platform-wide

9. Scalability & Flexibility Considerations

- Custom roles per property: the schema (Section 6.3.1) already supports property-scoped roles alongside global system roles, so enabling the Role & Permission Configuration Dashboard's "create fully custom role" capability later requires no migration.
- Granular, extensible permission engine: new permission codes (e.g. a future module) are just new rows in permissions plus corresponding role_permissions grants — no schema change, and hasPermission() automatically supports them.
- Multi-property staff (future): the current UNIQUE(property_id, mobile_number) design allows the same person to hold independent staff accounts at multiple properties today; a true "single identity, multiple property memberships" model can be introduced later via a join table (user_property_memberships) without breaking existing data, since users already carries a nullable property_id.
- Pluggable authentication: password_hash/pin_hash columns are deliberately generic; adding SSO/OAuth for Super Admin or corporate-chain Owners later is an additive column (e.g. auth_provider, external_id), not a redesign.
- Horizontal scale of the backend: FastAPI is stateless per request (session state lives in user_sessions/JWTs, not server memory), so multiple API instances can sit behind a load balancer; PostgreSQL is the single source of truth and can be scaled with read replicas for the heavy-read dashboards in Section 7.
- Sync engine reusability: the outbox pattern (user_sync_log) used here is the same pattern the BRD implies for every other offline module (Bookings, Payments, Housekeeping, etc.) — implementing it once as a shared core service (lib/core/sync/) means this module's sync logic is a thin, reusable case rather than bespoke code.
- Dashboard growth: each dashboard in Section 7 is backed by its own read endpoint, so adding a 7th dashboard later (e.g. a Franchise/Regional Manager rollup) is additive and does not touch the other five.

10. Security Considerations (Module-Specific)

Directly mapped from the BRD's platform-wide Security section, scoped to what this module is responsible for implementing:

- Local database encryption — `pin_hash`, `password_hash`, and any cached `PermissionSet` data are encrypted at rest on-device.
- Biometric login with PIN fallback — both paths funnel into the same offline authentication logic (Section 4.2); neither is a separate, weaker code path.
- Device registration and activation — every login is tied to a registered device via `user_devices`; unregistered devices cannot bootstrap offline credentials (Section 6.3.5).
- Role-based permissions — enforced identically client-side (UX) and server-side (real boundary), per Section 4.1.
- Remote device deactivation — Super Admin/Support can revoke a device's sessions the moment it reconnects, per Section 4.10.
- Audit logs for all critical actions — every `create/edit/deactivate/role-change/reset/lockout` event in this module writes an `audit_logs` row; the table is append-only at the database-permission level (Section 6.3.10).
- Secure backups — user/role data included in the platform's secure backup routine (owned by the Support Portal, this module supplies the data).

11. Summary

This blueprint gives your team everything needed to start building the User & Role Management module for Pinesphere Stay: the exact hierarchy and roles the BRD specifies, a permission model expressive enough to match the BRD's Role Matrix precisely, offline-first logics for authentication and sync, a normalised and constraint-safe database design across both PostgreSQL and the local SQLite/ObjectBox store, six purpose-built dashboards, and a phased (not date-bound) implementation path using Flutter and FastAPI. Every design decision traces back to a specific line in the BRD, and every extension point (custom roles, new permissions, multi-property staff, SSO) has been planned for without requiring rework of what you build first.